

X3DH Protocol Implementation

Comprehensive Code Execution & Theoretical
Guide

Python Cryptography Walkthrough

Extended Triple Diffie-Hellman Key Agreement

1. Introduction to X3DH

The **Extended Triple Diffie-Hellman (X3DH)** protocol is a fundamental cryptographic handshake used in modern secure messaging systems (like Signal and WhatsApp). It allows two parties (commonly referred to as Alice and Bob) to establish a shared secret key securely, even if Bob is offline. This is known as "asynchronous key agreement."

The Core Objectives of X3DH

- **Mutual Authentication:** Alice and Bob verify each other's identity to prevent Man-in-the-Middle (MITM) attacks.
- **Forward Secrecy:** If a long-term key is compromised in the future, past messages remain secure.
- **Cryptographic Deniability:** Neither party can mathematically prove to a third party that the other participated in the communication.

Keys Used in the Protocol Infrastructure

To achieve these properties, X3DH utilizes a combination of long-term identity keys, medium-term signed keys, and single-use ephemeral keys.

Party	Key Name	Symbol	Cryptographic Purpose
Alice	Identity Key	IKA	Long-term identity authentication. Proves Alice is who she says she is.
Alice	Ephemeral Key	EKA	Single-use key generated by Alice just for this specific session. Provides Forward Secrecy.
Bob	Identity Key	IKB	Long-term identity authentication. Proves Bob is who he says he is.
Bob	Signed Prekey	SPKB	Medium-term key uploaded to the server, signed by IKB. Mitigates the risk of a compromised server feeding fake keys to Alice.
Bob	One-Time Prekey	OPKB	Single-use key deleted immediately after use. Provides "Ultimate Forward Secrecy" for the initial message.

2. Execution & Code Mapping

This section maps the Python code execution to the theoretical steps of the protocol, explaining exactly what is happening under the hood.

Step 2.1: Key Generation (Preliminaries)

Before any communication occurs, both Alice and Bob must generate their long-term cryptographic identities. In this implementation, we use **Curve25519** for Diffie-Hellman exchanges and **Ed25519** for digital signatures.

PYTHON SOURCE CODE

```
print("\nGENERATING LONG-TERM & PREKEYS\n")
print("-> Generating IKA (Alice) and IKB (Bob)...")

IKA_private = x25519.X25519PrivateKey.generate()
IKA_public = IKA_private.public_key()

IKB_private = x25519.X25519PrivateKey.generate()
IKB_public = IKB_private.public_key()

IKB_sign_private = ed25519.Ed25519PrivateKey.generate()
IKB_sign_public = IKB_sign_private.public_key()
```

GENERATED CONSOLE OUTPUT

GENERATING LONG-TERM & PREKEYS

```
-> Generating IKA (Alice) and IKB (Bob)...
    Generating Alice Identity Key... [ALG: Curve X25519 Key Gen]
    Generating Bob Identity Key... [ALG: Curve X25519 Key Gen]
    Generating Bob Signing Identity Key... [ALG: Curve Ed25519 Key Gen]
```

Step 2.2: Phase 1 - Bob Publishes Prekeys

Because Bob might be offline when Alice wants to message him, he pre-generates keys and uploads them to a central server. He creates a **Signed Prekey (SPKB)**, which he signs with his long-term Identity Key (IKB). He also uploads a batch of **One-Time Prekeys (OPKB)**.

Why does Bob sign his Prekey?

If the central server is malicious (or hacked), it could give Alice a fake prekey belonging to an attacker. Because Bob signs his prekey (SPKB) with his identity key (IKB), Alice can verify the signature. Since only Bob has the private half of IKB, Alice can trust that the prekey genuinely belongs to Bob.

PYTHON SOURCE CODE

```
print("\nPHASE 1: BOB PUBLISHES PREKEYS TO SERVER\n")

SPKB_private = x25519.X25519PrivateKey.generate()
SPKB_public = SPKB_private.public_key()

# Bob signs the SPKB using his Ed25519 Identity Key
spkb_encoded = Encode(SPKB_public)
signature = IKB_sign_private.sign(spkb_encoded)

OPKB_private = x25519.X25519PrivateKey.generate()
OPKB_public = OPKB_private.public_key()
```

GENERATED CONSOLE OUTPUT

```
PHASE 1: BOB PUBLISHES PREKEYS TO SERVER

-> Bob generates signed prekey SPKB... [ALG: Curve X25519 Key Gen]

-> Bob generates prekey signature Sig(IKB, Encode(SPKB))... [ALG: Ed25519
Signature Gen]
    [Hexadecimal Signature Output]

-> Bob generates one-time prekey OPKB... [ALG: Curve X25519 Key Gen]

-> Bob publishes a set of elliptic curve public keys to the server.
```

Step 2.3: Phase 2 - Alice Initiates the Handshake

Alice wants to message Bob. She downloads Bob's "Prekey Bundle" from the server. After successfully verifying Bob's Ed25519 signature, she generates a fresh **Ephemeral Key (EKA)** and performs four distinct Elliptic Curve Diffie-Hellman (ECDH) calculations.

The Mathematical Core: The 4 DH Calculations

The "Triple" in X3DH comes from the first three of these equations. The fourth is the "Extended" part. By combining these, Alice builds a mathematically unbreakable shared secret.

- **DH1 = DH(IKA, SPKB):** Proves Alice's identity to Bob.
- **DH2 = DH(EKA, IKB):** Proves Bob's identity to Alice and provides forward secrecy.
- **DH3 = DH(EKA, SPKB):** Provides forward secrecy in case the long-term keys are compromised.
- **DH4 = DH(EKA, OPKB):** Uses the one-time key. If the medium-term SPKB is ever cracked, DH4 ensures the message remains secure because the OPKB is deleted immediately after use.

These four outputs are concatenated and run through a Key Derivation Function (**HKDF-SHA256**) to extract a clean, cryptographically secure 32-byte Shared Secret (SK). She uses this SK to encrypt her message using AES-256-GCM.

PYTHON SOURCE CODE

```
# Alice verifies the prekey signature...
IKB_sign_public.verify(bundle_signature, Encode(bundle_SPKB))

DH1_alice = DH(IKA_private, bundle_SPKB)
DH2_alice = DH(EKA_private, bundle_IKB)
DH3_alice = DH(EKA_private, bundle_SPKB)
DH4_alice = DH(EKA_private, bundle_OPKB)

# SK = KDF(DH1 || DH2 || DH3 || DH4)
KM_alice = DH1_alice + DH2_alice + DH3_alice + DH4_alice
SK_alice = KDF(KM_alice)

# Encrypt Payload
aesgcm_alice = AESGCM(SK_alice)
nonce = os.urandom(12)
ciphertext = aesgcm_alice.encrypt(nonce, b"Hello Bob!", AD)
```

GENERATED CONSOLE OUTPUT

PHASE 2: ALICE FETCHES BUNDLE & INITIATES

```
-> Alice contacts the server and fetches a 'prekey bundle'...
-> Alice verifies the prekey signature... [ALG: Ed25519 Signature Verification]
    Verification SUCCESS: Alice verifies the prekey signature.

-> Alice generates an ephemeral key pair with public key EKA...

-> Alice calculates DH1, DH2, DH3, and DH4... [ALG: 4x X25519 ECDH Operations]
    DH1: [Hex] | DH2: [Hex] | DH3: [Hex] | DH4: [Hex]

-> Alice calculates SK = KDF(DH1 || DH2 || DH3 || DH4)... [ALG: HKDF-SHA256]
    [32-byte Hex String Secret Key]

-> Alice encrypts the initial ciphertext using an AEAD scheme...
    Ciphertext: [Hexadecimal Encrypted Payload]
```

Step 2.4: Phase 3 - Bob Receives & Decrypts

Bob receives the encrypted payload from Alice. Attached to the payload are Alice's public Identity Key (IKA) and her Ephemeral Key (EKA). Bob uses his own private keys and Alice's public keys to compute the mathematical reciprocals of Alice's Diffie-Hellman operations.

Diffie-Hellman Reciprocity

Because of the algebraic properties of elliptic curves, Bob calculating $DH(SPKB_private, IKA_public)$ yields the exact same byte sequence as Alice calculating $DH(IKA_private, SPKB_public)$. Therefore, when Bob sums his four DH outputs and passes them through HKDF, he derives the exact same Shared Secret (SK) that Alice generated, allowing him to successfully decrypt the AES-GCM payload.

Immediately after decryption, Bob **deletes his One-Time Prekey (OPKB) private key**. This action guarantees ultimate forward secrecy for this conversation.

PYTHON SOURCE CODE

```
# Bob calculates reciprocal DH parameters
DH1_bob = DH(SPKB_private, received_IKA)
DH2_bob = DH(IKB_private, received_EKA)
DH3_bob = DH(SPKB_private, received_EKA)
DH4_bob = DH(OPKB_private, received_EKA)

# Bob derives SK
KM_bob = DH1_bob + DH2_bob + DH3_bob + DH4_bob
SK_bob = KDF(KM_bob)

# Bob decrypts the ciphertext
aesgcm_bob = AESGCM(SK_bob)
decrypted = aesgcm_bob.decrypt(nonce, ciphertext, AD_bob)

# Forward Secrecy Trigger
del OPKB_private
```

GENERATED CONSOLE OUTPUT

PHASE 3: BOB RECEIVES AND DECRYPTS

```
-> Bob receives message and retrieves Alice's IKA and EKA.

-> Bob repeats the DH calculations to derive SK... [ALG: 4x Reciprocal X25519]
-> Bob derives SK from combined DH segments... [ALG: HKDF-SHA256 Derivation]
    Bob's derived SK: [Identical 32-byte Hex String Secret Key]

-> Bob constructs the AD byte sequence using IKA and IKB.

-> Bob attempts to decrypt the initial ciphertext using SK and AD...

-> SUCCESS: The initial ciphertext decrypts successfully.
-> DECRYPTED MESSAGE: 'Hello Bob! This is an encrypted asynchronous payload...'
-> Bob deletes the one-time prekey (OPKB) private key for forward secrecy.
```

HANDSHAKE COMPLETE

⚠ **Security Disclaimer:**

This implementation is intended for educational, research, and academic demonstration purposes only. It maps out the theoretical steps of the X3DH specification but is not production-ready. It lacks robust state management, replay attack protections, and production-level key storage mechanisms. Do not use in real-world secure messaging systems without further hardening.